

Two Pointers M Container With Most Water

Container With Most Water : Given array height where each element represents vertical line height at that index, find two lines that together with x-axis form container holding maximum water (area = width $\times$ min(height)), Example Input: [1,8,6,2,5,4,8,3,7] $\rightarrow$ Output: 49; optimal approach uses two pointers at ends calculating area and moving pointer pointing to shorter line inward (because limiting factor is smaller height), Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Understanding how to iterate through and access array elements by index
- **Two Pointers Technique** - Moving pointers from opposite ends of an array to efficiently search for optimal pairs

2. Two Pointers

Intuition

Using two pointers lets us efficiently search for the maximum area without checking every pair.

We start with the widest container (left at start, right at end).

The height is limited by the shorter line, so to potentially increase the area, we must move the pointer at the shorter line inward.

Moving the taller line never helps because it keeps the height the same but reduces the width.

By always moving the shorter side, we explore all meaningful possibilities.

Algorithm

1. Initialize two pointers:
 - $l = 0$
 - $r = \text{len}(\text{heights}) - 1$
2. Set $\text{res} = 0$ to store the maximum area.
3. While $l < r$:
 - Compute the current area:
 $\text{area} = \min(\text{heights}[l], \text{heights}[r]) * (r - l)$
 - Update res with the maximum area so far.
 - Move the pointer at the shorter height:
 - If $\text{heights}[l] \leq \text{heights}[r]$, move l right.
 - Otherwise, move r left.
4. Return res after the pointers meet.

class Solution:

```
def maxArea(self, heights: List[int]) -> int:
    l, r = 0, len(heights) - 1
    res = 0

    while l < r:
        area = min(heights[l], heights[r]) * (r - l)
        res = max(res, area)
        if heights[l] <= heights[r]:
            l += 1
        else:
            r -= 1
    return res
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

func maxArea(heights []int) int {

```
    l, r := 0, len(heights) - 1
    res := 0

    for l < r {
        area := min(heights[l], heights[r]) * (r - l)
        if area > res {
            res = area
        }
        if heights[l] <= heights[r] {
            l++
        } else {
            r--
        }
    }
    return res
}
```

```
func min(a, b int) int {
    if a < b {
        return a
    }
    return b
}
```

Common Pitfalls

Moving the Wrong Pointer

The algorithm requires moving the pointer at the shorter height inward. Moving the taller pointer instead never increases the area (since height is limited by the shorter side) and can cause the algorithm to miss the optimal solution.

Confusing This Problem with Trapping Rain Water

Unlike the Trapping Rain Water problem where you need to sum water trapped between bars, this problem finds a single container formed by two lines. Applying the wrong mental model leads to incorrect area calculations.

Off-by-One Errors in Width Calculation

The width between indices l and r is $r - l$, not $r - l + 1$. Using the wrong formula overestimates the area by one unit for every pair, leading to incorrect results.

Two Pointers M 3Sum

3Sum : Given array nums, return all unique triplets such that sum equals zero (triplet means three distinct indices), Example Input: $[-1, 0, 1, 2, -1, -4] \rightarrow$ Output: $[[[-1, -1, 2], [-1, 0, 1]]]$; optimal approach sorts array then fixes one element and uses two pointers for remaining two while skipping duplicates to avoid repeated triplets, Time Complexity $O(n^2)$, Space Complexity $O(1)$ excluding output.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Sorting** - Sorting the array enables efficient duplicate handling and two-pointer technique
- **Two pointers** - The optimal solution uses two pointers to find pairs that sum to a target
- **Hash maps** - An alternative approach uses hash maps for $O(1)$ lookups
- **Handling duplicates** - Skipping duplicate values to avoid repeated triplets in the result

3. Two Pointers

Intuition

After sorting the array, we can fix one number and then search for the other two using the two-pointer technique.

Sorting helps in two ways:

1. It lets us skip duplicates easily.
2. It ensures that moving the left or right pointer will increase or decrease the sum in a predictable way.

For each fixed number a , we place two pointers:

- l starts just after i ,
- r starts at the end.

If the current sum is too large, we move r left to reduce it.

If the sum is too small, we move l right to increase it.

When the sum is exactly zero, we record the triplet and skip duplicates.

Algorithm

1. Sort the array to handle duplicates and enable two-pointer logic.
2. Loop through the array using index i :
 - Let $a = \text{nums}[i]$.
 - If $a > 0$, break (all remaining numbers are positive).
 - Skip duplicate values for the first number.
3. Set two pointers:
 - $l = i + 1$
 - $r = \text{len}(\text{nums}) - 1$
4. While $l < r$:
 - Compute $\text{threeSum} = a + \text{nums}[l] + \text{nums}[r]$.
 - If $\text{threeSum} > 0$, move r left.
 - If $\text{threeSum} < 0$, move l right.
 - If $\text{threeSum} == 0$:
 - Add the triplet to the result.
 - Move both pointers inward.
 - Skip duplicates at the left pointer.
5. Return the list of all valid triplets.

class Solution:

```
def threeSum(self, nums: List[int]) -> List[List[int]]:
```

```

res = []
nums.sort()

for i, a in enumerate(nums):
    if a > 0:
        break

    if i > 0 and a == nums[i - 1]:
        continue

    l, r = i + 1, len(nums) - 1
    while l < r:
        threeSum = a + nums[l] + nums[r]
        if threeSum > 0:
            r -= 1
        elif threeSum < 0:
            l += 1
        else:
            res.append([a, nums[l], nums[r]])
            l += 1
            r -= 1
            while nums[l] == nums[l - 1] and l < r:
                l += 1

    return res

```

Expand

Time & Space Complexity

- Time complexity: $O(n^2)$
- Space complexity:
 - $O(1)$ or $O(n)$ extra space depending on the sorting algorithm.
 - $O(m)$ space for the output list.

Where m is the number of triplets and n is the length of the given array.

```

func threeSum(nums []int) [][]int {
    res := [][]int{}
    sort.Ints(nums)

    for i := 0; i < len(nums); i++ {
        a := nums[i]
        if a > 0 {
            break
        }
        if i > 0 && a == nums[i-1] {
            continue
        }

        l, r := i+1, len(nums)-1
        for l < r {
            threeSum := a + nums[l] + nums[r]
            if threeSum > 0 {
                r--
            } else if threeSum < 0 {
                l++
            } else {
                res = append(res, []int{a, nums[l], nums[r]})
                l++
                r--
                for l < r && nums[l] == nums[l-1] {
                    l++
                }
            }
        }
    }

    return res
}

```

Common Pitfalls

Forgetting to Skip Duplicates

A common mistake is not properly skipping duplicate values, which leads to duplicate triplets in the result. After sorting, when you find a valid triplet, you must skip over all identical values for the first element (outer loop) and the left pointer. Failing to do this causes wrong answers on inputs like `[-1, -1, 0, 1, 1]`.

Not Sorting the Array First

The two-pointer approach only works correctly on a sorted array. Without sorting, moving pointers based on sum comparisons does not guarantee you will find all valid triplets. Always sort the input array before applying the two-pointer technique.

Incorrect Early Termination

When the first element `nums[i]` is positive, all remaining elements are also positive (since the array is sorted), so no triplet can sum to zero. However, incorrectly breaking when `nums[i] >= 0` instead of `nums[i] > 0` misses cases like `[0, 0, 0]`. The break condition should be `nums[i] > 0`, not `nums[i] >= 0`.

Two Pointers E Valid Palindrome

Valid Palindrome : Given a string, determine if it reads the same forward and backward after removing non-alphanumeric characters (letters and digits) and ignoring case (case-insensitive comparison means 'A' equals 'a'), Example Input: `s = "A man, a plan, a canal: Panama"` → Output: `true`; optimal approach uses two pointers starting at left and right, skipping non-alphanumeric characters and comparing lowercase characters until they cross, Time Complexity $O(n)$ because each character is visited at most once, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers** - Using pointers from both ends of a string to compare characters efficiently
- **String Manipulation** - Filtering characters, converting case, and reversing strings
- **Character Classification** - Identifying alphanumeric vs non-alphanumeric characters

2. Two Pointers

Intuition

Instead of building a new string, we can check the palindrome directly in-place using two pointers.

One pointer starts at the beginning (`l`) and the other at the end (`r`).

We move both pointers inward, skipping any characters that are not letters or digits.

Whenever both pointers point to valid characters, we compare them in lowercase form.

If at any point they differ, the string is not a palindrome.

This method avoids extra space and keeps the logic simple and efficient.

Algorithm

1. Initialize two pointers:
 - `l` at the start of the string,
 - `r` at the end of the string.
2. While `l` is less than `r`:
 - Move `l` forward until it points to an alphanumeric character.
 - Move `r` backward until it points to an alphanumeric character.
 - Compare the lowercase characters at `l` and `r`:
 - If they don't match, return false.
 - Move both pointers inward: `l += 1`, `r -= 1`.
3. If the loop finishes without mismatches, return true.

class Solution:

```
def isPalindrome(self, s: str) -> bool:
    l, r = 0, len(s) - 1

    while l < r:
        while l < r and not self.alphaNum(s[l]):
            l += 1
        while r > l and not self.alphaNum(s[r]):
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l, r = l + 1, r - 1
    return True

def alphaNum(self, c):
    return (ord('A') <= ord(c) <= ord('Z') or
            ord('a') <= ord(c) <= ord('z') or
            ord('0') <= ord(c) <= ord('9'))
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```

func isPalindrome(s string) bool {
    l, r := 0, len(s)-1

    for l < r {
        for l < r && !isAlphaNum(rune(s[l])) {
            l++
        }
        for r > l && !isAlphaNum(rune(s[r])) {
            r--
        }
        if unicode.ToLower(rune(s[l])) != unicode.ToLower(rune(s[r])) {
            return false
        }
        l++
        r--
    }
    return true
}

```

```

func isAlphaNum(c rune) bool {
    return unicode.IsLetter(c) || unicode.IsDigit(c)
}

```

Common Pitfalls

Not Skipping Non-Alphanumeric Characters

The problem requires ignoring all characters that are not letters or digits. Forgetting to skip spaces, punctuation, and special characters will cause false negatives. For example, "A man, a plan, a canal: Panama" should be recognized as a palindrome, but including the spaces and punctuation in the comparison will incorrectly return false.

Case Sensitivity

Letters must be compared in a case-insensitive manner. Comparing 'A' directly with 'a' will return false even though they should be treated as equal. Always convert both characters to the same case (lowercase or uppercase) before comparing.

Two Pointers M Two Sum II Input Array Is Sorted

Two Sum II – Input Array Is Sorted : Given sorted array and target, return 1-based indices of two numbers adding to target, Example Input: numbers=[2,7,11,15], target=9 → Output: [1,2]; optimal approach uses two pointers from start and end adjusting based on sum comparison (if sum too small move left forward, if too large move right backward), Time Complexity O(n), Space Complexity O(1).

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers technique** - Using pointers at both ends of a sorted array and moving them based on sum comparison
- **Binary Search** - Efficiently finding a target value in a sorted array in O(log n) time
- **Hash Maps** - Storing values and indices for O(1) lookup of complements

4. Two Pointers

Intuition

Because the array is sorted, we can use two pointers to adjust the sum efficiently. If the current sum is too big, moving the right pointer left makes the sum smaller. If the sum is too small, moving the left pointer right makes the sum larger. This lets us quickly close in on the target without checking every pair.

Algorithm

1. Initialize two pointers:
 - l = 0 (start)
 - r = len(numbers) - 1 (end)
2. While l < r:
 - Compute curSum = numbers[l] + numbers[r].
 - If curSum > target, move r left to reduce the sum.
 - If curSum < target, move l right to increase the sum.
 - If curSum == target, return [l + 1, r + 1].
3. If no pair matches the target, return an empty list.

class Solution:

```

def twoSum(self, numbers: List[int], target: int) -> List[int]:
    l, r = 0, len(numbers) - 1

```

```

while l < r:
    curSum = numbers[l] + numbers[r]

    if curSum > target:
        r -= 1
    elif curSum < target:
        l += 1
    else:
        return [l + 1, r + 1]
return []

```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```

func twoSum(numbers []int, target int) []int {
    l, r := 0, len(numbers) - 1

    for l < r {
        curSum := numbers[l] + numbers[r]
        if curSum > target {
            r--
        } else if curSum < target {
            l++
        } else {
            return []int{l + 1, r + 1}
        }
    }
    return []int{}
}

```

Common Pitfalls

Returning 0-Indexed Instead of 1-Indexed Results

The problem explicitly requires 1-indexed positions in the output. A common mistake is returning $[i, j]$ instead of $[i + 1, j + 1]$. Always double-check the problem requirements for indexing conventions, as returning 0-indexed results will be marked as incorrect.

Not Leveraging the Sorted Property

Since the array is already sorted, the two-pointer approach achieves $O(n)$ time with $O(1)$ space. Using a hash map still works but wastes the sorted property and uses $O(n)$ extra space unnecessarily. While not incorrect, failing to recognize and use the sorted property results in a suboptimal solution.

Moving Both Pointers in the Same Direction

In the two-pointer approach, when the sum is too large you should move the right pointer left, and when the sum is too small you should move the left pointer right. A common error is moving both pointers in the same direction or moving the wrong pointer, which causes the algorithm to miss the valid pair or loop infinitely.

Two Pointers M Find The Duplicate Number

Find The Duplicate Number : Given array nums containing $n+1$ integers where each integer in range $[1, n]$, find duplicate without modifying array and using constant extra space, Example Input: $[1, 3, 4, 2, 2]$ → Output: 2; optimal approach uses Floyd's Cycle Detection treating array indices as linked list pointers, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Array Manipulation** - Working with array indices and understanding value-to-index mappings
- **Hash Sets** - Using sets for $O(1)$ lookup to detect duplicates
- **Floyd's Cycle Detection** - The fast and slow pointer technique for detecting cycles in linked list-like structures
- **Binary Search on Value Range** - Applying binary search to count-based problems rather than sorted arrays

7. Fast And Slow Pointers

Intuition

Treat the array like a **linked list**, where each index points to the next index given by its value.

Because one number is duplicated, two indices will point into the **same chain**, creating a **cycle** — exactly like a linked list with a loop.

Using Floyd's **Fast & Slow Pointer** technique:

1. The **slow** pointer moves one step at a time.
2. The **fast** pointer moves two steps at a time.
3. If there's a cycle, they will eventually meet.

Once they meet, we start a new pointer from the beginning:

- Move both pointers one step at a time.

- The point where they meet again is the **duplicate number** (the entry point of the cycle).

Algorithm

1. Initialize two pointers slow = 0 and fast = 0.
2. Move:
 - slow = nums[slow]
 - fast = nums[nums[fast]]
 - until slow == fast.
3. Start a new pointer slow2 = 0.
4. Move both:
 - slow = nums[slow]
 - slow2 = nums[slow2]
 - until they meet.
5. The meeting point is the duplicate number.
6. Return that number.

class Solution:

```
def findDuplicate(self, nums: List[int]) -> int:
    slow, fast = 0, 0
    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break

    slow2 = 0
    while True:
        slow = nums[slow]
        slow2 = nums[slow2]
        if slow == slow2:
            return slow
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```
func findDuplicate(nums []int) int {
    slow, fast := 0, 0
```

```
    for {
        slow = nums[slow]
        fast = nums[nums[fast]]

        if slow == fast {
            break
        }
    }
```

```
    slow2 := 0
    for {
        slow = nums[slow]
        slow2 = nums[slow2]

        if slow == slow2 {
            return slow
        }
    }
}
```

Common Pitfalls

Modifying the Array When Not Allowed

Some problem variants require not modifying the input array. Solutions like negative marking or in-place sorting alter the original data. Always check the constraints before choosing an approach that mutates the input.

Off-by-One Errors in Index Mapping

Since values range from 1 to n but array indices start at 0, forgetting to subtract 1 when mapping values to indices causes out-of-bounds errors or incorrect results. For example, value n maps to index n-1, not index n.

Misunderstanding Floyd's Cycle Detection Entry Point

In the fast and slow pointer approach, the meeting point of the two pointers is not the duplicate number. After they meet, you must start a new pointer from index 0 and move both pointers one step at a time until they meet again. This second meeting point is the cycle entry, which corresponds to the duplicate value.

Two Pointers M Remove Duplicates From Sorted Array II

Remove Duplicates From Sorted Array II : Given sorted array, allow each element to appear at most twice, Example Input: [1,1,1,2,2,3] → Output: 5 with array [1,1,2,2,3]; optimal approach uses slow pointer and ensures $\text{nums}[\text{slow}-2] \neq$ current element before placing, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers** - Using a read pointer and write pointer to modify arrays in-place
- **In-Place Array Modification** - Overwriting elements without using extra space
- **Sorted Array Properties** - Leveraging the fact that duplicates are always adjacent in sorted arrays

4. Two Pointers (Optimal)

Intuition

The cleanest approach uses a single condition: we only write an element if the write position is less than 2 (first two elements always go through) OR the current element differs from the element two positions back in the result. This automatically limits each value to at most two occurrences.

Algorithm

1. Initialize the write pointer l at 0.
2. Iterate through each element in the array.
3. If $l < 2$ or the current element differs from $\text{nums}[l - 2]$, write it at position l and increment l .
4. Return l as the new length.

class Solution:

```
def removeDuplicates(self, nums: List[int]) -> int:
    l = 0
    for num in nums:
        if l < 2 or num != nums[l - 2]:
            nums[l] = num
            l += 1
    return l
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$ extra space.

func removeDuplicates(nums []int) int {

```
    l := 0
    for _, num := range nums {
        if l < 2 || num != nums[l-2] {
            nums[l] = num
            l++
        }
    }
    return l
}
```

Common Pitfalls

Comparing with the Wrong Element

In the optimal two-pointer solution, you should compare $\text{nums}[i]$ with $\text{nums}[l - 2]$, not with $\text{nums}[l - 1]$. Comparing with $l - 1$ only checks if the current element differs from the previous one, which doesn't prevent more than two consecutive duplicates. You need to look back two positions to ensure at most two occurrences of any value.

Forgetting the Base Case

Arrays with 2 or fewer elements are already valid (each element can appear at most twice). Some solutions fail to handle these edge cases, leading to index out of bounds errors when checking $\text{nums}[l - 2]$ with $l < 2$. Always either return early for small arrays or use a condition like $l < 2$ to bypass the comparison.

Modifying the Wrong Position

When using two pointers, the write pointer l should advance only after writing a valid element. A common bug is incrementing l before writing or writing to the wrong index. The pattern should be: check condition, write to $\text{nums}[l]$, then increment l . Writing to $\text{nums}[l++]$ in languages that support it keeps this atomic.

Two Pointers M Rotate Array

Rotate Array : Given array nums and integer k , rotate array to right by k steps (rotation means shifting elements cyclically), Example Input: $\text{nums}=[1,2,3,4,5,6,7]$, $k=3$ → Output: $[5,6,7,1,2,3,4]$; optimal approach uses array reversal algorithm (reverse whole array, reverse first k , reverse remaining), Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Array Manipulation** - Understanding how to modify arrays in-place by swapping and shifting elements
- **Modular Arithmetic** - Using the modulo operator to handle rotation amounts larger than array length
- **Array Reversal** - The technique of reversing a subarray in-place using two pointers
- **Cyclic Traversal** - Following cycles in an array where each element points to its destination position

4. Using Reverse

Intuition

A clever observation: rotating an array by k is equivalent to moving the last k elements to the front. We can achieve this with three reversals. First, reverse the entire array. Now the last k elements are at the front, but in reverse order. Reverse the first k elements to fix their order. Finally, reverse the remaining elements to restore their original order.

Algorithm

1. Compute $k = k \% n$.
2. Reverse the entire array from index 0 to $n - 1$.
3. Reverse the first k elements (from index 0 to $k - 1$).
4. Reverse the remaining elements (from index k to $n - 1$).

class Solution:

```
def rotate(self, nums: List[int], k: int) -> None:
    """
    Do not return anything, modify nums in-place instead.
    """
    n = len(nums)
    k %= n

    def reverse(l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l, r = l + 1, r - 1

    reverse(0, n - 1)
    reverse(0, k - 1)
    reverse(k, n - 1)
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$ extra space.

```
func rotate(nums []int, k int) {
    n := len(nums)
    k %= n

    reverse := func(l, r int) {
        for l < r {
            nums[l], nums[r] = nums[r], nums[l]
            l++
            r--
        }
    }

    reverse(0, n-1)
    reverse(0, k-1)
    reverse(k, n-1)
}
```

Common Pitfalls

Forgetting to Handle k Greater Than Array Length

When k is larger than the array length n , rotating by k is the same as rotating by $k \% n$. Failing to normalize k with the modulo operation leads to unnecessary iterations or index out of bounds errors. Always compute $k = k \% n$ before proceeding.

Incorrect Reversal Boundaries in the Three-Reverse Approach

In the reverse approach, the three reversals must use precise boundaries: reverse the entire array, then reverse $[0, k-1]$, then reverse $[k, n-1]$. Using wrong indices like reversing $[0, k]$ instead of $[0, k-1]$ shifts the boundary incorrectly and produces a wrong result.

Modifying Array While Iterating Without Proper Tracking

In the cyclic replacement approach, failing to track how many elements have been placed can cause infinite loops or incomplete rotations. This happens when n and k share a common divisor, creating multiple cycles. Always maintain a counter and start new cycles until all elements are moved.

Two Pointers E Merge Strings Alternately

Merge Strings Alternately : Given two strings word1 and word2, merge them by alternating characters starting from word1, Example Input: word1="abc", word2="pqr" → Output: "apbqcr"; optimal approach uses two pointers iterating over both strings and appending characters sequentially until one finishes then append remainder, Time Complexity $O(n+m)$, Space Complexity $O(n+m)$ for result string.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Strings** - Understanding string indexing and character access
- **Two Pointers** - Using multiple indices to traverse data structures simultaneously
- **StringBuilder / String Concatenation** - Efficiently building strings in loops to avoid $O(n^2)$ complexity

3. One Pointer

Intuition

Since we always process characters at the same index from both strings in each iteration, we can simplify to a single index variable. We iterate up to the length of the longer string, and for each index, we add the character from each string if that index is valid.

Algorithm

1. Let n and m be the lengths of word1 and word2.
2. Initialize an empty result list.
3. For each index i from 0 to $\max(n, m) - 1$:
 - If $i < n$, append word1[i] to the result.
 - If $i < m$, append word2[i] to the result.
4. Return the joined result string.

class Solution:

```
def mergeAlternately(self, word1: str, word2: str) -> str:
    n, m = len(word1), len(word2)
    res = []
    for i in range(max(m, n)):
        if i < n:
            res.append(word1[i])
        if i < m:
            res.append(word2[i])
    return "".join(res)
```

Time & Space Complexity

- Time complexity: $O(n+m)O(n+m)$
- Space complexity: $O(n+m)O(n+m)$ for the output string.

Where nn and mm are the lengths of the strings word1word1 and word2word2 respectively.

func mergeAlternately(word1 string, word2 string) string {

```
    n, m := len(word1), len(word2)
    var res strings.Builder

    maxlen := n
    if m > maxlen {
        maxlen = m
    }

    for i := 0; i < maxlen; i++ {
        if i < n {
            res.WriteByte(word1[i])
        }
        if i < m {
            res.WriteByte(word2[i])
        }
    }

    return res.String()
}
```

Common Pitfalls

Forgetting to Append the Remaining Characters

When one string is longer than the other, the remaining characters must be appended after the alternating portion is complete. Stopping the loop when the shorter string ends without handling the leftover characters produces an incomplete result.

Using String Concatenation in a Loop

In many languages, repeatedly concatenating strings with + inside a loop creates a new string object each time, leading to $O(n^2)$ time complexity. Use a StringBuilder, list of characters, or similar efficient structure to build the result, then join at the end.

Two Pointers E Move Zeroes

Move Zeroes : Given array `nums`, move all zeros to end while maintaining relative order of non-zero elements (stable operation means order preserved), Example Input: `[0,1,0,3,12]` → Output: `[1,3,12,0,0]`; optimal approach uses slow pointer for next non-zero placement and fast pointer to scan array, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Understanding array indexing and in-place modification
- **Two Pointers** - Using two pointers to partition or rearrange elements within an array

3. Two Pointers (One Pass)

Intuition

Instead of copying values and then filling zeros separately, we can swap elements in a single pass. The left pointer marks the boundary between processed non-zero elements and unprocessed elements. When we encounter a non-zero element with the right pointer, we swap it with the element at the left pointer. This naturally pushes 0 to the right while keeping non-zero elements in their relative order.

Algorithm

1. Initialize `l = 0` to track the swap position.
2. Iterate through the array with pointer `r`. For each non-zero element:
 - Swap `nums[l]` and `nums[r]`.
 - Increment `l`.
3. After the loop, all non-zero elements are at the front and zeros are at the end.

class Solution:

```
def moveZeroes(self, nums: List[int]) -> None:
    """
    Do not return anything, modify nums in-place instead.
    """
    l = 0
    for r in range(len(nums)):
        if nums[r]:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

func moveZeroes(nums []int) {

```
    l := 0
    for r := 0; r < len(nums); r++ {
        if nums[r] != 0 {
            nums[l], nums[r] = nums[r], nums[l]
            l++
        }
    }
}
```

Common Pitfalls

Swapping When Pointers Are at the Same Position

When both pointers `l` and `r` point to the same non-zero element, swapping is unnecessary and wastes operations. While this does not affect correctness, adding a check like `if (l != r)` before swapping improves efficiency and avoids redundant writes, which can matter for performance-sensitive applications.

Disrupting Relative Order of Non-Zero Elements

The problem requires maintaining the relative order of non-zero elements. Some approaches incorrectly swap non-zero elements with each other or move them out of sequence. The two-pointer technique works because the left pointer only advances when a non-zero element is placed, ensuring all non-zero elements shift left in their original order while zeros naturally accumulate at the end.

Two Pointers E Merge Sorted Array

Merge Sorted Array : Given two sorted arrays `nums1` and `nums2`, merge `nums2` into `nums1` in-place assuming `nums1` has enough extra space at end, Example Input: `nums1=[1,2,3,0,0,0]`, `m=3`; `nums2=[2,5,6]`, `n=3` → Output: `nums1` becomes `[1,2,2,3,5,6]`; optimal approach uses three pointers starting from end (largest elements) to avoid overwriting values, Time Complexity $O(n+m)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Arrays** - Understanding how to access and modify elements by index
- **Two Pointers** - Traversing arrays from different directions simultaneously
- **In-place Algorithms** - Modifying data structures without using extra space

4. Three Pointers Without Extra Space - II

Intuition

This is a cleaner version of the previous approach. We observe that once all elements from `nums2` are placed, the remaining elements of `nums1` are already in their correct positions. So we only need to loop while `j >= 0`. This simplifies the logic and removes the need for a separate cleanup loop.

Algorithm

1. Initialize pointers `i = m - 1`, `j = n - 1`, and `last = m + n - 1`.
2. While `j >= 0`:
 - If `i >= 0` and `nums1[i] > nums2[j]`, place `nums1[i]` at `nums1[last]` and decrement `i`.
 - Otherwise, place `nums2[j]` at `nums1[last]` and decrement `j`.
 - Decrement `last`.

class Solution:

```
def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
    """
    Do not return anything, modify nums1 in-place instead.
    """
    last = m + n - 1
    i, j = m - 1, n - 1

    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[last] = nums1[i]
            i -= 1
        else:
            nums1[last] = nums2[j]
            j -= 1
        last -= 1
```

Expand

Time & Space Complexity

- Time complexity: $O(m+n)$
- Space complexity: $O(1)$ extra space.

Where `m` and `n` represent the number of elements in the arrays `nums1` and `nums2`, respectively.

```
func merge(nums1 []int, m int, nums2 []int, n int) {
    last := m + n - 1
    i, j := m-1, n-1

    for j >= 0 {
        if i >= 0 && nums1[i] > nums2[j] {
            nums1[last] = nums1[i]
            i--
        } else {
            nums1[last] = nums2[j]
            j--
        }
        last--
    }
}
```

Common Pitfalls

Merging From the Front Instead of the Back

When merging in place, starting from the front of `nums1` overwrites elements that have not yet been processed. This destroys data you still need. Always merge from the back of `nums1` where there is empty space, placing the largest elements first.

Forgetting to Copy Remaining Elements From `nums2`

After the main merge loop, if there are remaining elements in `nums2`, they must be copied to `nums1`. Elements remaining in `nums1` are already in their correct positions, but leftover `nums2` elements need explicit placement. Missing this step leaves the result incomplete.

Remove Duplicates From Sorted Array : Given sorted array (array arranged in non-decreasing order), remove duplicates in-place such that each element appears once and return new length, Example Input: [1,1,2] → Output: 2 with array [1,2]; optimal approach uses two pointers where slow pointer tracks last unique element, Time Complexity $O(n)$, Space Complexity $O(1)$.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Two Pointers** - Using a read pointer and write pointer to modify arrays in-place
- **In-Place Array Modification** - Overwriting elements without using extra space
- **Sorted Array Properties** - Understanding that duplicates are always adjacent in sorted arrays

3. Two Pointers - II

Intuition

A more elegant approach: we compare each element with its predecessor. Since duplicates are consecutive in a sorted array, an element is unique if it differs from the one before it. We maintain a write pointer that only advances when we find a new unique value.

Algorithm

1. Start with $l = 1$ since the first element is always unique.
2. Iterate r from 1 to the end of the array.
3. If $nums[r]$ differs from $nums[r - 1]$, it's a new unique value.
4. Copy it to position l and increment l .
5. Return l as the count of unique elements.

class Solution:

```
def removeDuplicates(self, nums: list[int]) -> int:
    l = 1
    for r in range(1, len(nums)):
        if nums[r] != nums[r - 1]:
            nums[l] = nums[r]
            l += 1
    return l
```

Time & Space Complexity

- Time complexity: $O(n)O(n)$
- Space complexity: $O(1)O(1)$

```
func removeDuplicates(nums []int) int {
    l := 1
    for r := 1; r < len(nums); r++ {
        if nums[r] != nums[r-1] {
            nums[l] = nums[r]
            l++
        }
    }
    return l
}
```

Common Pitfalls

Not Leveraging the Sorted Property

The array is already sorted, meaning duplicates are always adjacent. Some solutions use a hash set or sort the array again, which wastes time and space. Since duplicates are consecutive, you only need to compare each element with its predecessor (or the last written element) to detect duplicates in $O(1)$ space.

Returning the Wrong Value

The function should return the count of unique elements, not modify and return the array itself. Additionally, the returned length k means the first k elements of $nums$ contain the unique values. Some solutions off-by-one error by returning $l - 1$ instead of l , or forget that the write pointer already represents the count of unique elements written.

Two Pointers M Max Number of K-Sum Pairs

Max Number of K-Sum Pairs: Given array and integer k , return maximum operations removing pairs summing to k , approach use hash map counting complements or sort and use two pointers, optimal time $O(n)$ with map, space $O(n)$.

Example:

Input: $nums=[1,2,3,4]$, $k=5$

Output: 2
